

OPERACIÓN KANTABILE

PROYECTO INTERMODULAR

ADRIÁN VICENTE LÓPEZ

EDUARDO PIQUER SANCHEZ

-CONTENIDO-

1. Introducción	3
2. Descripción del proyecto	3
2.1 Usuarios del sistema	3
2.2 Funcionalidades para usuarios	4
2.3 Funcionalidades para administradores	4
3. Objetivos del proyecto	5
4. Tecnologías utilizadas	5
5. Organización del equipo	6
6. Planificación inicial	6
7. Arquitectura del sistema	7
8. Estado actual y viabilidad	7
9. Modelo de base de datos	7
8.1 Integridad y Persistencia	8
8.2 Relaciones y esquema E/R	8

1. Introducción

Este documento presenta la planificación y el estado actual del desarrollo de **Kantabile**, una página web de karaoke. El objetivo del proyecto es crear una plataforma en la que los usuarios puedan reproducir canciones con vídeo y letra sincronizada, ofreciendo una experiencia similar a la de un karaoke tradicional pero adaptada al entorno web.

La aplicación permitirá a los usuarios visualizar las canciones disponibles en el sistema y añadirlas a una cola de reproducción personal. Cada usuario dispondrá de su propio reproductor y podrá gestionar su propia cola de canciones, permitiendo añadir nuevas canciones o modificar el orden de reproducción según sus preferencias. De esta forma, **Kantabile** funciona como un karaoke personal que puede utilizarse de forma individual o en grupo con amigos.

Además, los usuarios tendrán la posibilidad de solicitar nuevas canciones mediante una página de peticiones. Estas solicitudes podrán ser revisadas posteriormente por los administradores del sistema para decidir si se añaden al catálogo de canciones disponibles.

Cada usuario dispondrá también de una página de perfil desde la que podrá gestionar su información personal, como modificar su contraseña u otros datos de su cuenta.

Por otro lado, el sistema contará con un panel de administración desde el cual los administradores podrán gestionar las canciones disponibles, revisar y administrar las peticiones de nuevas canciones y gestionar los usuarios registrados en la plataforma. Los administradores también podrán acceder al reproductor y utilizar la aplicación como usuarios normales si lo desean.

En este documento se describen los objetivos de **Kantabile**, las tecnologías utilizadas, la organización del trabajo y el estado actual del desarrollo.

2. Descripción del proyecto

La aplicación consiste en una plataforma web de karaoke que permite a los usuarios reproducir canciones con vídeo y letra sincronizada. El sistema está diseñado para que cada usuario pueda disfrutar de una experiencia de karaoke personal, gestionando su propia cola de reproducción y seleccionando las canciones que desea interpretar.

La plataforma se divide en dos tipos principales de usuarios: usuarios normales y administradores, cada uno con diferentes funcionalidades dentro del sistema.

2.1 Usuarios del sistema

El sistema cuenta con dos tipos de usuarios:

Usuarios normales

Son los usuarios que utilizan la plataforma para disfrutar del karaoke. Estos usuarios pueden acceder al reproductor, gestionar su cola de canciones y realizar peticiones de nuevas canciones que aún no se encuentren en el sistema.

Administradores

Los administradores son los encargados de gestionar el contenido y el funcionamiento del sistema. Tienen acceso a herramientas de administración que les permiten añadir nuevas canciones, gestionar usuarios y revisar las peticiones realizadas por los usuarios. Además, los administradores también pueden utilizar el reproductor y el resto de funcionalidades como si fueran usuarios normales.

2.2 Funcionalidades para usuarios

Los usuarios registrados en la plataforma disponen de diferentes funcionalidades que les permiten interactuar con el sistema.

Reproductor de karaoke

Los usuarios pueden acceder a una página de reproductor donde se muestran las canciones disponibles en el sistema. Desde esta página pueden añadir canciones a su cola de reproducción personal. El reproductor se encarga de reproducir las canciones seleccionadas mostrando el vídeo correspondiente junto con la letra sincronizada.

Gestión de la cola de canciones

Cada usuario dispone de su propia cola de reproducción. Dentro de esta cola puede añadir canciones, eliminar canciones o modificar el orden en el que se reproducirán.

Página de peticiones

Si un usuario desea cantar una canción que no está disponible en el sistema, puede realizar una solicitud desde la página de peticiones para que dicha canción sea añadida al catálogo.

Perfil de usuario

Cada usuario cuenta con una página de perfil desde la que puede consultar y modificar su información personal, como por ejemplo su contraseña u otros datos de su cuenta.

2.3 Funcionalidades para administradores

Los administradores disponen de un panel de gestión que les permite administrar el contenido del sistema.

Gestión de canciones

Los administradores pueden añadir nuevas canciones al sistema, así como editar o eliminar canciones existentes.

Gestión de usuarios

Desde el panel de administración se pueden gestionar los usuarios registrados en la plataforma, permitiendo consultar información, modificar datos o realizar otras acciones de administración.

Gestión de peticiones

Los administradores pueden revisar las peticiones de canciones realizadas por los usuarios. A partir de estas solicitudes pueden decidir si añadir nuevas canciones al sistema.

3. Objetivos del proyecto

Objetivo general

Desarrollar una plataforma web que permita ofrecer una experiencia de karaoke accesible desde el navegador, facilitando la reproducción de canciones y la gestión del contenido del sistema.

Objetivos específicos

Diseñar una interfaz sencilla e intuitiva para los usuarios.

Implementar un sistema que permita organizar y reproducir canciones.

Facilitar la incorporación de nuevas canciones mediante un sistema de peticiones.

Desarrollar herramientas de administración para la gestión del contenido.

Garantizar una experiencia de uso fluida y accesible para los usuarios.

4. Tecnologías utilizadas

Para el desarrollo de la aplicación web se han seleccionado tecnologías y herramientas que permiten un desarrollo ágil, flexible y compatible con la mayoría de dispositivos. A continuación se detallan las principales tecnologías empleadas.

Entorno de desarrollo

VSCode: editor de código utilizado para programar tanto el frontend como el backend.

XAMPP: servidor local con Apache y MySQL que permite ejecutar y probar la aplicación durante el desarrollo.

Frontend

HTML y CSS: para estructurar y diseñar la interfaz web.

Bootstrap: framework de CSS que facilita la creación de interfaces responsivas y consistentes.

JavaScript: se utilizará de forma complementaria para añadir interactividad y mejorar la experiencia del usuario cuando sea necesario.

Backend

PHP: lenguaje de servidor utilizado para procesar las peticiones del usuario, gestionar la base de datos y controlar la lógica de la aplicación.

Base de datos

MySQL: sistema de gestión de base de datos relacional que almacena información de usuarios, canciones, peticiones y otros datos relevantes del sistema.

Control de versiones y colaboración

Git y GitHub: el control de versiones se gestiona mediante Git, utilizando VSCode para interactuar con GitHub. Esto permite centralizar el código, gestionar revisiones y facilitar la colaboración entre los miembros del equipo de manera organizada.

Herramientas de apoyo

Durante el desarrollo se han utilizado herramientas de inteligencia artificial como apoyo en ciertas tareas de programación y documentación.

La combinación de estas tecnologías y herramientas permite crear un sistema web funcional, escalable y compatible con distintos dispositivos, asegurando un desarrollo colaborativo, organizado y ágil.

5. Organización del equipo

El desarrollo de **Kantabile** se realiza en un equipo de dos integrantes. Debido al tamaño reducido del equipo, ambos participantes colaboran de manera flexible en todas las partes del proyecto, incluyendo el diseño de la base de datos, la interfaz web y la lógica de la aplicación.

Esta forma de trabajo permite distribuir las tareas según las necesidades del proyecto y avanzar de manera ágil, realizando revisiones constantes del trabajo de manera colaborativa.

6. Planificación inicial

El desarrollo de **Kantabile** se organiza de forma **flexible y colaborativa**, adaptándose a la disponibilidad del equipo y a las necesidades que vayan surgiendo durante el proyecto. Aunque no se dispone de un cronograma rígido, sí se ha definido un conjunto de **tareas principales** que estructuran el trabajo y facilitan el avance progresivo del sistema:

Tareas principales

Diseño y creación de la base de datos.

Desarrollo de la interfaz de usuario, incluyendo las páginas del reproductor, perfil y peticiones e interfaz de administración.

Implementación de la lógica de usuario y de administrador.

Gestión y administración de canciones y peticiones.

Integración del reproductor con vídeo y letra sincronizada.

Pruebas, ajustes y mejoras de la experiencia de usuario.

Gestión del trabajo y seguimiento

El equipo mantiene un **registro de tareas pendientes y completadas**, permitiendo priorizar actividades y coordinar el trabajo de manera eficiente. Este enfoque flexible asegura que el proyecto avance de forma constante, incluso adaptándose a cambios o necesidades que puedan surgir durante el desarrollo.

7. Arquitectura del sistema

La arquitectura de **Kantabile** se basa en un modelo **cliente-servidor**. Los usuarios interactúan con la plataforma a través del navegador web, utilizando su reproductor personal y gestionando su cola de canciones.

El servidor, desarrollado en PHP, procesa las solicitudes de los usuarios y se comunica con la base de datos MySQL, que almacena información sobre usuarios, canciones y peticiones. El sistema incluye además una interfaz de administración que permite a los administradores gestionar canciones, revisar solicitudes y administrar usuarios.

Esta arquitectura modular y flexible facilita la integración de nuevas funcionalidades y asegura un desarrollo organizado y mantenible del proyecto.

8. Estado actual y viabilidad

Hasta el momento, el proyecto **Kantabile** cuenta con la estructura de base de datos definida y funcionando, así como con las páginas principales del sistema tanto para usuarios como para administradores.

La planificación flexible del equipo permite avanzar de manera continua, priorizando funcionalidades esenciales y dejando espacio para integrar características más complejas, como la sincronización de vídeo y letra en el reproductor.

La elección de tecnologías consolidadas (PHP, MySQL, Bootstrap y VSCode con GitHub) asegura que el proyecto sea **técnicamente viable** y que el equipo pueda continuar desarrollando de forma organizada.

9. Modelo de base de datos

El sistema utiliza un sistema de gestión de bases de datos relacionales (RDBMS) **MariaDB** gestionado mediante la interfaz **phpMyAdmin**. Esta elección permite mantener la integridad referencial entre los usuarios, el catálogo musical y el estado del sistema en tiempo real (cola de reproducción).

La base de datos, denominada **karaoke**, se compone de cuatro tablas interconectadas mediante claves foráneas (Foreign Keys):

Proyecto Intermodular-Operación kantabile

usuarios: Gestiona el acceso al sistema. Almacena el nombre, email, contraseña y el rol (Usuario o Administrador). Incluye un campo estado para el control de bloqueos de cuenta.

canciones: Es el núcleo del catálogo. No solo guarda metadatos (título, artista, estilo), sino que gestiona las rutas de sistema de archivos para los tres componentes esenciales del karaoke: el audio con **voz**, la pista **instrumental** y el archivo de **letra** (formato .lrc).

cola: Actúa como la tabla dinámica de la sesión. Relaciona qué usuario ha pedido qué canción, permitiendo gestionar el orden de actuación.

peticiones: Permite a los usuarios solicitar canciones que aún no están en el catálogo, incluyendo un control de estado (pendiente o realizado) y marca de tiempo.

8.1 Integridad y Persistencia

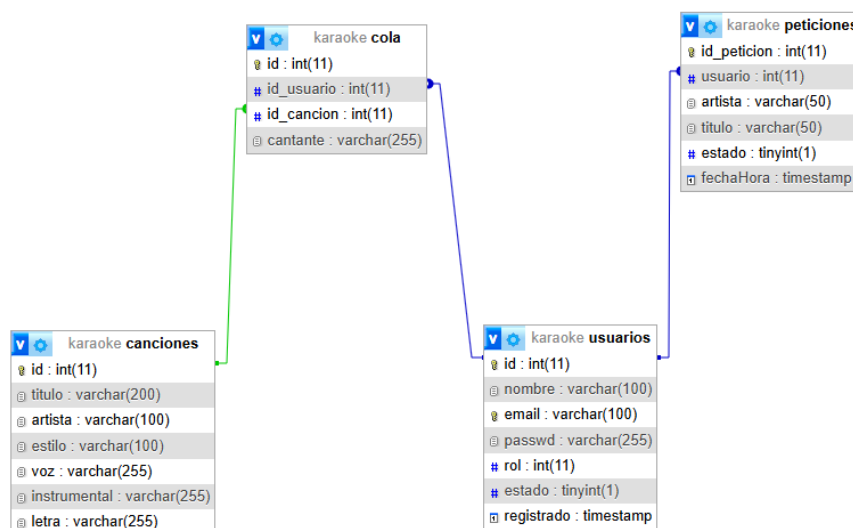
Se han implementado restricciones de integridad para garantizar que, si un usuario o canción se elimina, las entradas correspondientes en la cola o en peticiones se gestionen automáticamente (mediante ON DELETE CASCADE), evitando registros huérfanos.

8.2 Relaciones y esquema E/R

Usuarios → Cola: Una línea que salga de usuarios.id hacia cola.id_usuario. (Relación 1:N).

Canciones → Cola: Una línea que salga de canciones.id hacia cola.id_cancion. (Relación 1:N).

Usuarios → Peticiones: Una línea que salga de usuarios.id hacia peticiones.usuario. (Relación 1:N).



10. LÓGICA BASE

La lógica central del proyecto ha sido diseñada para actuar como un middleware inteligente entre la base de datos MariaDB y el sistema de archivos del servidor. El objetivo es garantizar que la biblioteca musical sea **autónoma, escalable y libre de errores de redundancia**.

10.1. Algoritmo de Organización y Normalización

El sistema no almacena los archivos de forma plana, sino que sigue una estructura jerárquica basada en metadatos para optimizar la búsqueda y el mantenimiento.

Normalización Estricta de Cadenas: Se utiliza la función `preg_replace('/[^\A-Za-z0-9\-\ ]/', '', $string)` para sanear los nombres. Esta técnica es fundamental para eliminar caracteres que el sistema de archivos podría interpretar como comandos o rutas (como `../` o símbolos de sistema), garantizando la portabilidad entre servidores Linux y Windows.

Tratamiento de Espacios: Se aplica `str_replace(' ', '_', $string)`. El uso de guiones bajos en lugar de espacios evita los errores de "espacios escapados" (`%20`) en las URLs, lo que facilita la carga directa de los recursos en el reproductor HTML5.

Prevención de Colisiones mediante Entropía Temporal: Se añade un timestamp dinámico a cada archivo. Esto asegura que, si dos versiones de la misma canción son subidas, el sistema genere nombres únicos, evitando la pérdida de datos por sobrescritura accidental.

10.2. Estructura de Directorios Dinámica

El sistema implementa una arquitectura de almacenamiento por nodos de artista, gestionada mediante las siguientes funciones nativas de PHP:

Verificación de Existencia (`is_dir`): Antes de cada operación de escritura, el script comprueba si la rama del árbol de directorios ya existe, evitando errores de duplicidad.

Creación Recursiva (`mkdir`): Si el artista es nuevo en el catálogo, se ejecuta `mkdir($ruta, 0777, true)`. El parámetro `true` es crítico, ya que permite la creación de carpetas anidadas de forma recursiva en un solo paso, mientras que el permiso `0777` asegura que el servidor tenga privilegios de lectura/escritura sobre los nuevos **assets**.

Distribución de Componentes: Se segmentan los recursos en `/uploads/canciones/` para flujos binarios de audio y `/uploads/letras/` para archivos de texto sincronizado, manteniendo una separación de intereses clara en el servidor.

10.3. Gestión de la Integridad (CRUD de Archivos)

Se ha implementado una lógica de "borrado inteligente" para mantener una sincronización perfecta entre la base de datos y el almacenamiento físico (Estado Espejo).

Sincronización de Borrado (unlink): Al eliminar un registro, el sistema no solo borra la fila en SQL, sino que utiliza las rutas almacenadas para invocar la función unlink(). Esto libera espacio en el disco de forma inmediata.

Análisis de Residuos (scandir): Tras la eliminación de los archivos, se escanea el directorio del artista. El sistema cuenta los elementos devueltos por scandir(), ignorando los punteros de navegación de sistema (. y ..).

Recursividad Inversa de Limpieza (rmdir): Si el contador de archivos es cero, el sistema interpreta que el artista ya no tiene canciones en el catálogo y procede a eliminar la carpeta física con rmdir(). Esto evita la acumulación de carpetas vacías (directorios huérfanos) que degradarían el rendimiento del sistema de archivos a largo plazo.

10.4. Explicación Técnica de Funcionamiento (Workflow)

Para entender el proceso completo desde la interfaz hasta el disco, el flujo se divide en cuatro etapas técnicas:

Etapas de Captura: El formulario **multipart/form-data** envía los arrays globales \$_POST (datos) y \$_FILES (binarios). El sistema valida que los archivos no superen el límite de subida del servidor (**upload_max_filesize**).

Etapas de Saneamiento: Se limpian los nombres del artista y título. Se escapan las cadenas con **mysqli_real_escape_string()** para asegurar que caracteres como el apóstrofe (ej. "Guns N' Roses") no corrompan la sentencia SQL posterior.

Etapas de Persistencia Física: Se utiliza **move_uploaded_file()**. Esta función es una medida de seguridad vital en PHP, ya que verifica que el archivo que se intenta mover es realmente un archivo subido mediante HTTP POST y no un ataque de inclusión de archivos locales.

Etapas de Registro: Solo si los archivos se mueven correctamente al servidor, se ejecuta la consulta INSERT en MariaDB. Esto garantiza que no existan registros en la base de datos que apunten a archivos inexistentes.

10.5. Ejemplo del Flujo de Nombres

Concepto	Valor de Entrada	Valor Procesado
Canción	"Zafar"	Zafar
Artista	"La Vela Puerca"	La_Vela_Puerca
Timestamp	1711834200	1711834200

Resultado final en el Servidor:

uploads/canciones/La_Vela_Puerca/Zafar_(voz)_1711834200.mp3

uploads/canciones/La_Vela_Puerca/Zafar_(instrumental)_1711834200.mp3

uploads/letras/La_Vela_Puerca/Zafar_1711834200.lrc

Ejemplo de como se guardara las carpetas para las canciones y letras

```
// limpiamos los nombres (Evitamos espacios y caracteres raros en carpetas y archivos)
// "Rosalía - Motomami" -> "Rosalía_Motomami"
$artista_folder = str_replace(' ', '_', preg_replace('/[^A-Za-z0-9\_-]/', '', $artista));
$nombre_limpio = str_replace(' ', '_', preg_replace('/[^A-Za-z0-9\_-]/', '', $titulo));
```

Ejemplo de como se guardan los archivos de instrumental, voz y letra

```
//Creamos el nombre final (Título_Tiempo.extension)
// Añadimos time() al final por si subes dos veces la misma canción, que no se borren
$nombre_final_voz = $nombre_limpio . "_(" . date('s') . ".") . $ext_voz;
$nombre_final_instrumental = $nombre_limpio . "_(" . date('s') . ".") . $ext_instrumental;
$nombre_final_letra = $nombre_limpio . "_(" . date('s') . ".lrc";
```

Ejemplo de la creación de carpetas

```
/creamos las carpetas si no existen (con permisos totales para evitar problemas al subir archivos)
/ El 0777 es permisos totales, el 'true' permite crear carpetas anidadas
if (!is_dir($dir_canciones)) {
    mkdir($dir_canciones, 0777, true);
}
if (!is_dir($dir_letras)) {
    mkdir($dir_letras, 0777, true);
}
```

11. Motor de Ejecución y Gestión de Sesión

Este apartado describe la lógica central de "Kantabile", que gestiona la interacción en tiempo real entre el usuario, la base de datos y los flujos multimedia sincronizados.

11.1. Gestión Dinámica de Colas Privadas

El sistema implementa una arquitectura de cola personalizada mediante el uso de sesiones de PHP y consultas SQL filtradas por identificador de usuario (\$idUserio).

Aislamiento de Sesión: Mediante la instrucción WHERE id_usuario = '\$idUserio', el sistema garantiza que la experiencia sea multi-instancia, permitiendo que cada usuario gestione su propia lista de reproducción sin interferencias.

Gestión de Turnos: El sistema identifica automáticamente la canción activa mediante una consulta con LIMIT 1 y ordenación ascendente por ID, asegurando que el flujo de la sesión siga el orden cronológico de selección.

Control de Estado (Mover/Quitar): Se han desarrollado scripts de soporte (**cancion_mover.php**, **cancion_quitar.php**) que permiten al usuario reordenar su cola en tiempo real, modificando la prioridad de los registros en la base de datos.

11.2. Arquitectura del Reproductor Dual

Una de las innovaciones técnicas del proyecto es el uso de un sistema de audio sincronizado por software.

Dualidad de Pistas: El reproductor carga simultáneamente dos nodos de audio HTML5: la pista de Voz (guía) y la pista Instrumental.

Sincronización de Flujos: Mediante JavaScript, se ha vinculado el currentTime de ambas pistas. Al pausar, reproducir o desplazar la barra de progreso (evento onseeking), los dos archivos se mantienen en fase con una precisión de milisegundos.

Modo Guía Dinámico: Se implementó una función de mutado selectivo que permite al usuario alternar entre la versión con voz y la instrumental sin interrumpir la reproducción, facilitando el aprendizaje de la canción.

11.3. Motor de Sincronización Avanzada

El sistema utiliza un algoritmo de procesamiento en el lado del cliente para transformar archivos de texto .lrc en una experiencia visual interactiva.

Parsing de Micro-tiempos: A diferencia de reproductores básicos que solo detectan líneas, este motor utiliza Expresiones Regulares (RegExp) para detectar marcas de tiempo de frase [...] y marcas de palabra <...>. Esto permite un resaltado progresivo (tipo "bola de rebote") dentro de la misma frase.

Ciclo de Renderizado: En lugar de intervalos fijos, se utiliza el método requestAnimationFrame. Esto permite que el "engine" de sincronización se ejecute a la

tasa de refresco del monitor (normalmente 60fps), garantizando una transición suave en las barras de progreso de cada palabra mediante variables CSS.

Lógica de Pre-visualización: El motor calcula dinámicamente el índice de la frase actual y la siguiente, permitiendo al usuario anticipar la letra que debe cantar (campo linea-siguiente).

11.4. Interfaz de Escenario

Para profesionalizar la experiencia de usuario, se ha desarrollado un módulo de Escenario Externo.

Comunicación Inter-Ventana: Utilizando el objeto `window.open`, el sistema abre una ventana secundaria independiente del panel de control.

Sincronización del DOM: Mediante un puente de datos en JavaScript, la ventana principal inyecta el contenido procesado de las letras en la ventana del escenario en tiempo real. Esto permite que el monitor principal sea utilizado para la gestión (cola/búsqueda) mientras el monitor secundario muestra exclusivamente la letra al cantante, simulando un entorno de karaoke profesional.

12. Navegación y Flujo de Trabajo (User Flow)

En este apartado se describe la arquitectura de navegación de la aplicación, diseñada bajo un enfoque de experiencia de usuario (UX) simplificada para evitar distracciones en entornos de ocio (karaoke).

12.1. Mapa del Sitio (Sitemap)

La aplicación se estructura en cuatro nodos principales interconectados:

Index / Login: Puerta de entrada donde se gestiona la autenticación.

Dashboard (Biblioteca): El centro neurálgico donde el usuario busca canciones y gestiona su perfil.

Reproductor de Karaoke: Interfaz de ejecución donde se procesa el audio y la letra.

Ventana de Escenario: Una extensión visual independiente para la proyección de la letra.

12.2. Descripción de los Flujos Principales

Flujo de Selección de Canción

Para que un usuario pueda cantar, el sistema sigue este camino lógico:

Búsqueda: El usuario utiliza el buscador dinámico. El sistema filtra en tiempo real sobre la base de datos.

Registro en Cola: Al hacer clic en "Añadir a la cola", se dispara una petición POST que registra el `id_cancion` y el alias del cantante.

Carga de Assets: Cuando llega el turno, el sistema carga simultáneamente tres elementos: el audio guía, el audio instrumental y el archivo de texto .lrc.

Flujo de Control de Visualización

Una de las innovaciones de "Kantabile" es su flujo de visualización dual, pensado para monitores externos o proyectores:

Acción del Usuario: Desde el reproductor, el usuario pulsa "Modo Escenario".

Apertura de Nodo: JavaScript abre una ventana pop-up limpia (sin controles, solo letra).

Sincronización: El flujo de datos de la ventana principal se "espeja" en la secundaria. Esto permite que el gestor controle el volumen y la cola mientras el cantante solo ve la letra.

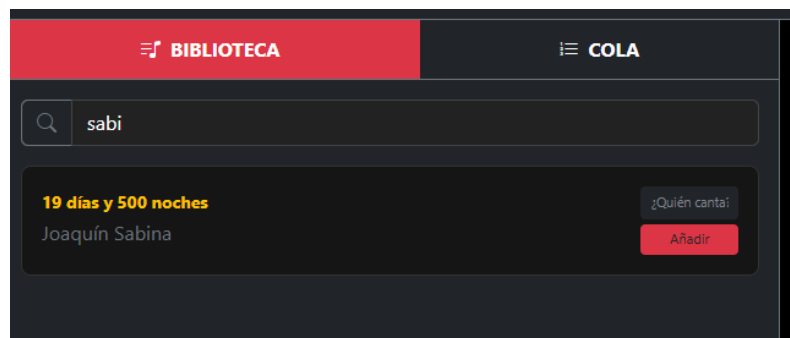
12.3. Diseño de la Interfaz

La navegación se ha diseñado para no ser intrusiva.

Sidebar Fijo: El menú lateral permite saltar entre la biblioteca y la cola de espera sin detener la música que esté sonando.

Feedback Visual: Se han implementado estados de "Cargando" y notificaciones de éxito cuando una canción se añade correctamente a la lista.

Un ejemplo visual de la interfaz sería el que veremos a continuación

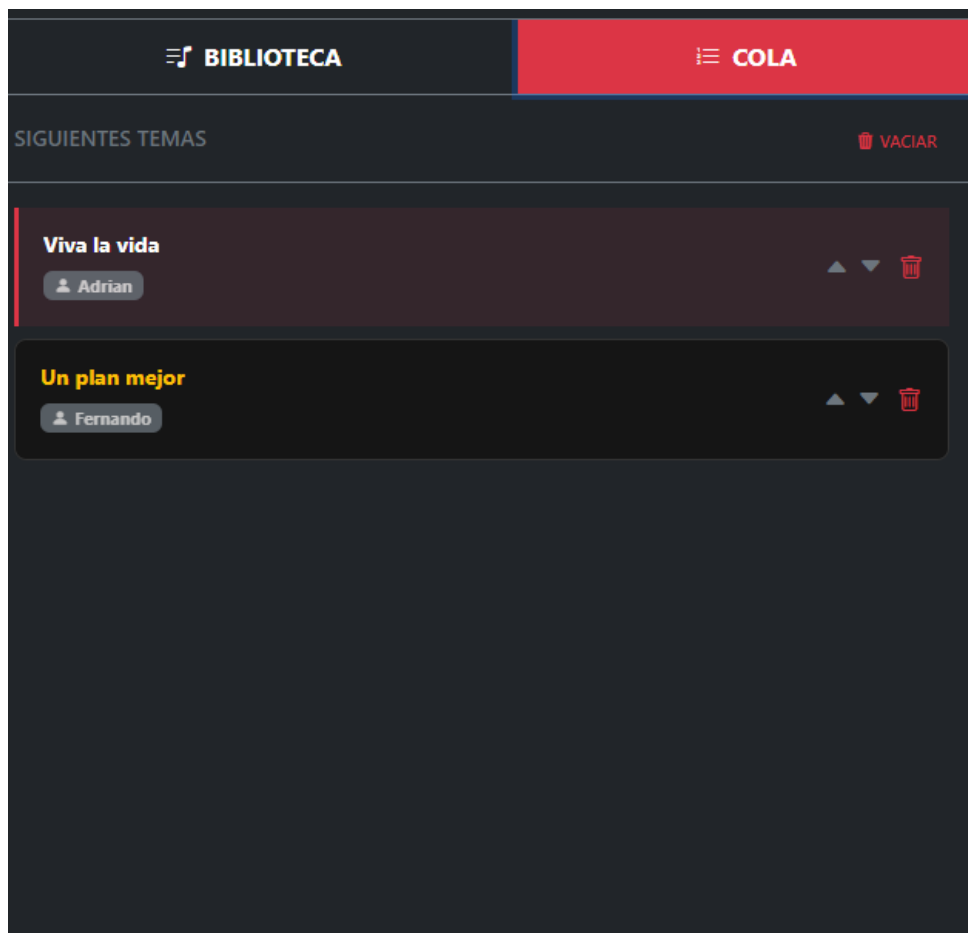
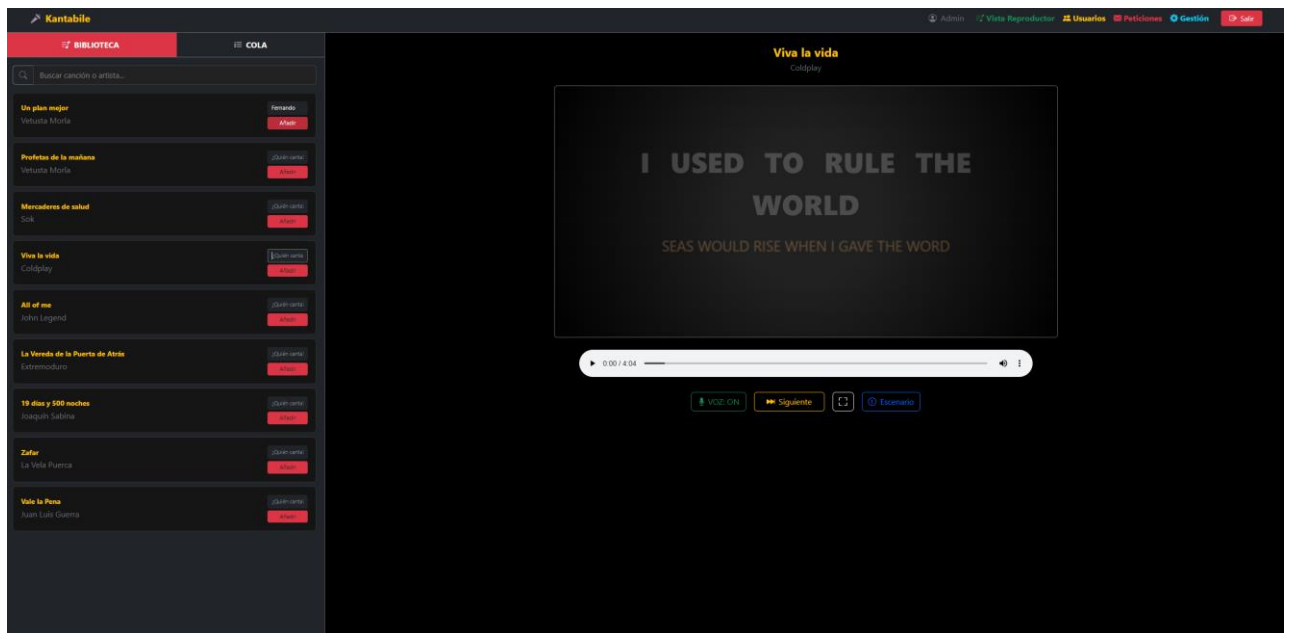


El buscador dinámico en esta imagen busca coincidencias basándose en artista, canción o estilo.

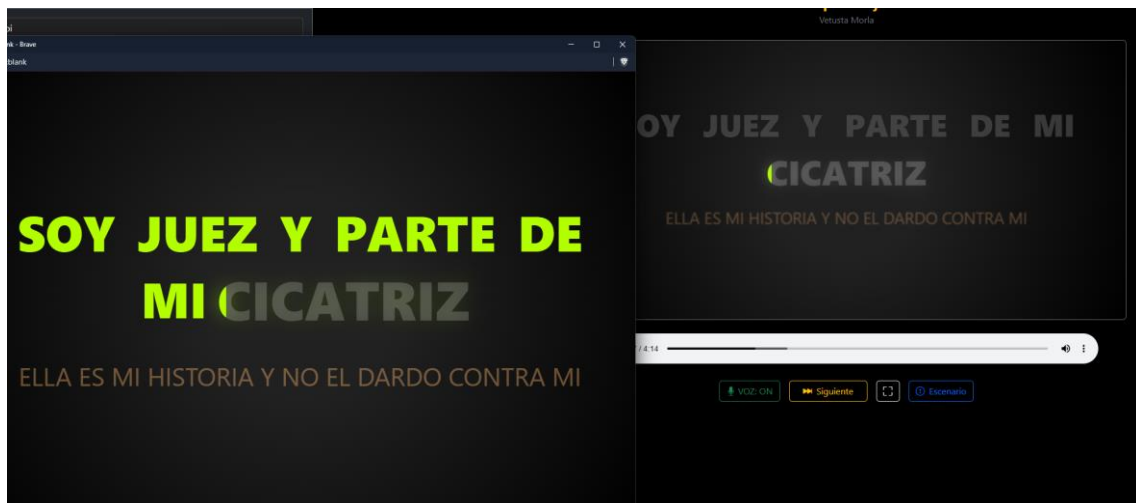
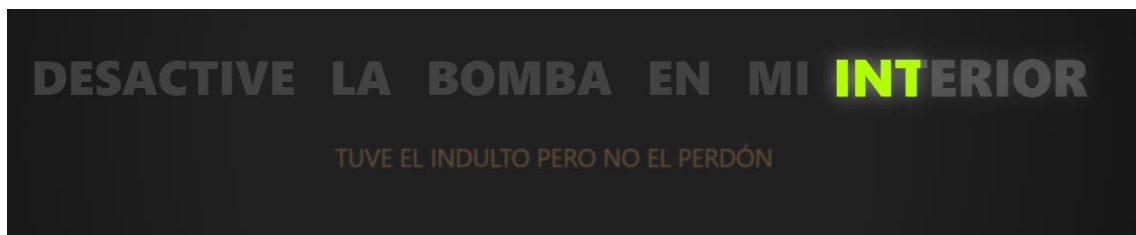
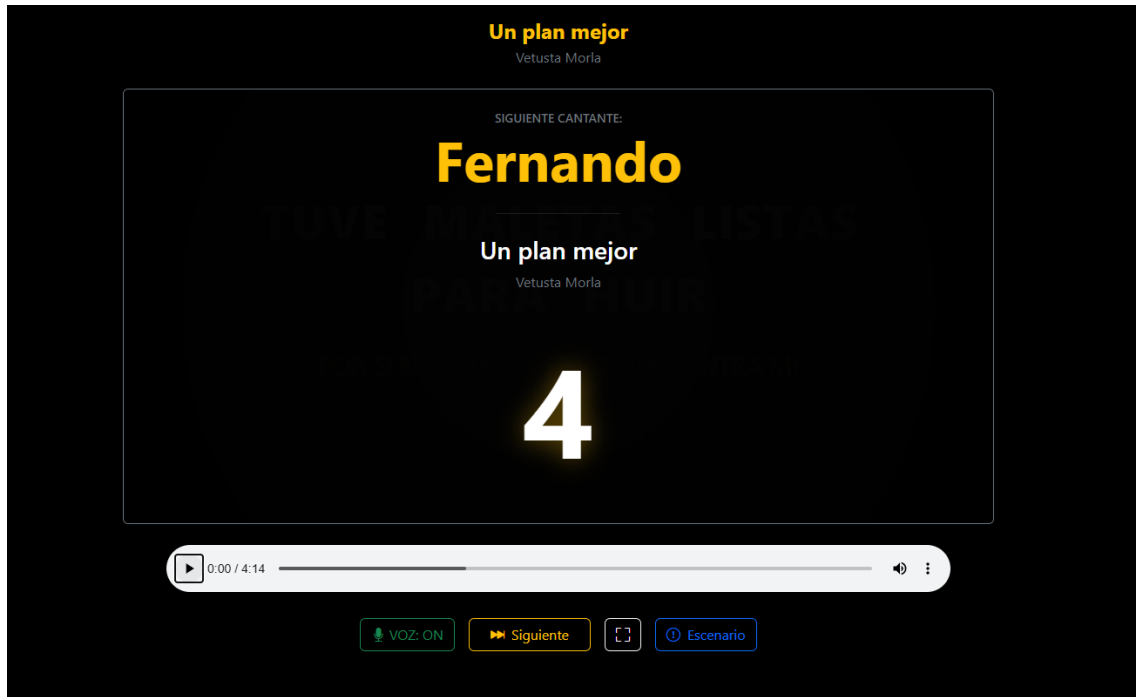
En la siguiente imagen se observará la interfaz completa de una sola pagina donde se ve mediante tabs tanto la biblioteca con un buscador y un input para el alias como la gestión de colas de reproducción.

A su vez en la derecha tenemos el reproductor, que es el que hace la magia de este proyecto.

Proyecto Intermodular-Operación kantabile



Proyecto Intermodular-Operación kantabile



13. Conectividad e Infraestructura de Comunicación

La robustez de "Kantabile" reside en la comunicación eficiente entre sus distintos componentes. A continuación, se describen los niveles de conectividad implementados:

13.1. Comunicación Cliente-Servidor (HTTP/API)

La interacción entre el front-end (navegador) y el back-end (PHP) se basa en el protocolo HTTP. Se han utilizado peticiones asíncronas para las siguientes tareas:

- ✓ Validación de credenciales.
- ✓ Consultas a la base de datos de canciones.
- ✓ Actualización de la cola de reproducción en tiempo real.

4.2. Gestión de Recursos Multimedia

La conectividad con el sistema de archivos del servidor es crítica para la reproducción. El sistema gestiona la carga de:

Flujos de Audio: Los archivos **.mp3** se cargan de forma progresiva mediante el elemento **<audio>** de HTML5.

Metadatos LRC: El archivo de letras se descarga como un recurso estático que el motor de JavaScript procesa localmente para garantizar una sincronización de 0 milisegundos de latencia.

4.3. Conectividad entre Contextos

Una característica técnica destacada es la conectividad entre la ventana principal y la ventana del escenario.

Protocolo de Memoria: No se utiliza una conexión de red para este proceso, sino una comunicación directa a través del DOM del navegador.

Sincronismo: El objeto `window.open` permite que la ventana maestra actúe como servidor de contenidos y la ventana esclava como receptor de renderizado, asegurando que la letra se vea exactamente igual en ambas pantallas sin desfase.